

Aula 04 — Fundamentos da Linguagem C Embarcada

Descrição: Estudo de sintaxe básica, tipos de dados, variáveis, operadores matemáticos/lógicos e boas práticas de código limpo em C.

Objetivo: Compreender a sintaxe essencial da linguagem C aplicada a microcontroladores, dominando o uso de tipos de dados, variáveis, operadores e as premissas de código limpo para sistemas com recursos restritos.

Bloco 1 — Introdução ao C para Sistemas Embarcados

1.1 O Papel do Compilador e o Fluxo de Execução

Na programação de microcontroladores (como a plataforma Arduino), o código escrito no computador é chamado de **sketch**. Esse código é redigido em um dialeto baseado na linguagem C/C++ (frequentemente associado à linguagem de programação *Wiring*). No entanto, microcontroladores não conseguem ler texto diretamente; eles compreendem apenas níveis lógicos de tensão e instruções em formato binário.

O **compilador** é o utilitário de software responsável por traduzir o código-fonte legível por humanos para o **código de máquina** (arquivos binários contendo sequências de 0s e 1s).

- Quando clicamos no botão **Verify (Verificar)** na IDE do Arduino, o compilador apenas analisa a sintaxe do programa para garantir que ele respeita rigorosamente as regras estruturais da linguagem C.
 - Ao clicar no botão **Upload (Carregar)**, o processo de compilação é concluído, o arquivo binário gerado é transferido através do cabo USB e gravado permanentemente na memória do chip de controle do hardware.
-

1.2 A Estrutura Básica de um Sketch: as Funções `setup()` e `loop()`

Um programa para microcontrolador obrigatoriamente deve conter duas funções fundamentais pré-definidas para que possa compilar com sucesso:

```
void setup() {  
    // Código executado apenas uma única vez  
}  
  
void loop() {  
    // Código executado repetidamente em ciclo infinito  
}
```

- **A Função `setup()`:** Executada apenas **uma única vez** no momento em que a placa é ligada, quando o botão *Reset* é pressionado ou após a transferência de um novo código. Destina-se às **configurações iniciais do sistema**, como a definição dos pinos do microcontrolador que atuarão como entrada ou saída (`pinMode`), inicialização de taxas de transmissão de barramentos seriais (`Serial.begin`) e acionamento inicial de periféricos.

- **A Função `loop()`:** Entra em execução sequencial imediatamente após o término de `setup()`. Seu comportamento é o de um **laço de repetição infinito (ciclo de amostragem)**. Ela executa as linhas de comando de cima para baixo, passo a passo; ao atingir a chave de fechamento `}`, o programa reinicia a leitura a partir da primeira instrução do bloco. É onde ocorrem as leituras contínuas de sensores, o processamento de lógica de controle e o acionamento dinâmico de atuadores.
-

1.3 Gerenciamento de Memória e Restrições de Hardware

Diferente de um computador pessoal, que dispõe de gigabytes de RAM e armazenamento, os microcontroladores operam sob severas limitações físicas de recursos computacionais:

1. **Memória Flash (Não Volátil):** Destina-se ao armazenamento do código binário do programa (o *sketch* compilado). Os dados aqui guardados não são perdidos quando o dispositivo é desligado.
 - No microcontrolador **ATmega328** (utilizado no Arduino Uno), há apenas **32 kB de Flash** (sendo que 2 kB são reservados para o *bootloader* de inicialização do chip, restando 30 kB úteis).
 - No **ATmega2560** (Arduino Mega), dispõe-se de **128 kB de Flash** (com 4 kB dedicados ao *bootloader*).
 2. **Memória SRAM (Estática de Acesso Aleatório — Volátil):** Utilizada como memória de trabalho para armazenar variáveis temporárias de processamento, pilhas de funções e dados de comunicação. É extremamente escassa:
 - O **ATmega328 possui somente 2 kB (2.000 bytes) de SRAM**.
 - **Consequência Crítica:** Se o programador utilizar tipos de dados muito grandes ou arrays gigantescos, a memória SRAM se esgotará rapidamente. **O Arduino não emite alertas de falta de memória durante a execução do programa; ele simplesmente trava ou reinicia sem aviso prévio.**
 - *Soluções de otimização:* Para economizar SRAM, podemos forçar o armazenamento de dados imutáveis (como fontes de caracteres ou mensagens longas de texto) diretamente na memória Flash utilizando o modificador especial **PROGMEM** (necessitando importar a biblioteca `<avr/pgmspace.h>`).
 3. **Memória EEPROM (Não Volátil e Gravável):** Utilizada para persistência de dados de calibração que devem sobreviver à falta de energia elétrica (como valores máximos ou configurações do usuário), mas que precisam ser alterados eventualmente por software.
-

1.4 Boas Práticas Iniciais: Código Limpo Embarcado

Desenvolver código limpo (*clean code*) em hardware de recursos restritos é essencial para a manutenção e estabilidade do projeto:

- **Nomenclatura de Variáveis (Convenção CamelCase):** Nomes de variáveis devem ser descritivos (mostrando claramente sua utilidade no circuito) e não conter espaços em branco ou caracteres especiais. A convenção amplamente recomendada dita que variáveis e funções iniciem com **letras minúsculas** e, caso o nome seja composto, as palavras seguintes comecem com **letras maiúsculas** (exemplo: `ledPin` ou `tempoEsperaAlerta`).

- **Indentação e Espaços em Branco:** O compilador ignora completamente recuos de tabulação (indentações), quebras de linha ou espaços redundantes. Eles servem exclusivamente para garantir a leitura humana. Sempre organize o código com chaves alinhadas para delimitar os escopos de forma legível.
- **Comentários Descritivos:** Servem para explicar o funcionamento e a intenção da lógica, sendo ignorados pelo compilador. Em C, utilizamos:
 - `//` para **comentários de linha única**.
 - `/*` e `*/` para **comentários multilinha ou blocos de código**.

Bloco 2 — Tipos de Dados e Variáveis

2.1 Tipos Primitivos na Arquitetura AVR (8 bits)

Uma variável é uma região nomeada da memória SRAM onde podemos armazenar informações durante o processamento do programa. Em C, é obrigatório declarar o **tipo exato de dado** que a variável armazenará. A tabela abaixo exibe os tipos primitivos na arquitetura de microcontroladores AVR (como o Arduino Uno):

Tipo de Dado	Tamanho na SRAM	Faixa de Valores	Aplicação Recomendada
boolean / bool	1 byte	true (1) ou false (0)	Flags de estado (ex: ledLigado = true).
char	1 byte	-128 a 127	Armazenar caracteres individuais ASCII (entre aspas simples <code>' '</code>).
byte	1 byte	0 a 255	Controle de pinos físicos e comunicação serial de bytes.
int	2 bytes (16 bits)	-32.768 a 32.767	Leituras de ADC (0 a 1023), cálculos e contadores pequenos.
unsigned int	2 bytes (16 bits)	0 a 65.535	Contadores estritamente positivos de até 16 bits.
long	4 bytes (32 bits)	-2.147.483.648 a 2.147.483.647	Cálculos matemáticos grandes.
unsigned long	4 bytes (32 bits)	0 a 4.294.967.295	Medições de tempo longas via função millis() .
float	4 bytes (32 bits)	$\pm 3.4028235 \times 10^{38}$	Números reais/ponto flutuante (cálculos de engenharia e física).
double	4 bytes (32 bits)	O mesmo que o float	No Arduino Uno, o double não possui precisão dupla de 8 bytes (é processado exatamente como float).

2.2 Modificadores de Tipo

Os modificadores de tipo (**signed**, **unsigned**, **short**, **long**) instruem o compilador sobre duas características centrais da alocação de memória: **a largura em bits reservada (espaço físico alocado na SRAM)** e a **interpretação semântica dos bits (gerenciamento do sinal algébrico)**.

- **signed:** Modo padrão. Indica que a variável pode receber tanto valores positivos quanto negativos, mantendo o bit mais significativo dedicado à representação do sinal algébrico.
- **unsigned:** Elimina o bit de sinal de variáveis do tipo **char** ou **int**, permitindo armazenar apenas números positivos e o zero. **Como consequência, dobra-se a capacidade positiva de representação de valor** (ex: o **char** vai de -128 a 127, enquanto o **unsigned char** vai de 0 a 255).
- **short:** Garante que uma variável inteira terá tamanho fixo de 16 bits.
- **long:** Expande o tamanho de variáveis inteiras para 32 bits (ou indica maior precisão para floats em sistemas não-AVR).

Por que esse conceito é crítico em Sistemas Embarcados?

O gerenciamento criterioso dos modificadores decorre de limitações físicas do hardware:

a) Economia de SRAM Estrita: Em chips como o ATmega328, existem apenas **2.048 bytes (2 kB) de SRAM**. Se pinos digitais, contadores simples ou leituras discretas forem armazenados usando **int** (2 bytes) em vez de **byte/unsigned char** (1 byte), gasta-se o dobro de memória desnecessariamente.

b) Custo Computacional da CPU: Sendo o ATmega uma CPU puramente de 8 bits, seus registradores internos operam com 1 byte por vez. Manipular uma variável de 32 bits (**long** ou **unsigned long**) exige que o compilador gere múltiplas instruções assembly em cascata para carregar, comparar ou somar cada pedaço do número. Portanto, usar o modificador adequado economiza ciclos de clock da CPU e melhora a velocidade de resposta do sistema.

c) Prevenção de Estouro de Variável (Overflow / Underflow): Quando uma variável atinge seu limite de representação, o bit de transporte (*carry*) é descartado ou invade o bit de sinal:

- Em um **signed int** com valor **32767**, somar **1** resulta em **-32768** (inversão catastrófica de sinal em sistemas de controle).
- Se o contador for estritamente positivo (como tempo de contagem ou leitura de encoders), declarar como **unsigned int** amplia o teto seguro para **65535** antes de reiniciar em **0**[cite: 1, 10].
- Variáveis de base temporal contínua (ex.: medições via função **millis()**) obrigatoriamente demandam **unsigned long**, garantindo contagem contínua sem números negativos até ultrapassar 4 bilhões de milissegundos (~49,7 dias)

2.3 Escopo de Variáveis: Locais versus Globais

O local de declaração de uma variável define a visibilidade (escopo) e o seu tempo de existência na memória do dispositivo:

```
int pinoSensor = A0; // VARIÁVEL GLOBAL: Visível em todo o código.
```

```
void loop() {  
    int leituraLocal = analogRead(pinoSensor); // VARIÁVEL LOCAL: Existe apenas no ciclo do loop().
```

```
    static int contadorBipes = 0; // LOCAL ESTÁTICA: Visível apenas no loop(), mas não perde seu valor.  
    contadorBipes++;
```

}

1. **Variáveis Globais:** Declaradas fora de qualquer função (no topo do sketch). Elas permanecem na memória SRAM durante todo o tempo de funcionamento da placa e podem ser acessadas e modificadas por qualquer função do programa.
 - o **Perigo elétrico/lógico:** O uso massivo de variáveis globais aumenta desnecessariamente o consumo contínuo da escassa RAM. Além disso, dificulta o diagnóstico de erros, já que qualquer trecho isolado do programa pode alterar o valor da variável de forma inesperada.
 2. **Variáveis Locais:** Declaradas dentro de um bloco delimitador (como o interior de uma função ou de um laço `for`). São criadas no momento em que a função entra em execução e destruídas imediatamente quando ela se encerra, liberando espaço de memória. Elas não podem ser vistas ou alteradas por outras funções externas.
 3. **Variáveis Locais Estáticas (`static`):** Se declararmos uma variável dentro de uma função precedida pelo qualificador `static`, ela se comportará como uma variável local (visível apenas dentro daquela função), mas **não será destruída ao fim da execução**. Ela reterá seu valor acumulado de modo que, na próxima vez que a função for chamada, o dado estará preservado, sem a necessidade de criar uma variável global.
-

2.4 Diretiva `#define` versus Qualificador `const`

A diretiva `#define` e o qualificador `const` são mecanismos utilizados na linguagem C para associar nomes legíveis a valores imutáveis, diferenciando-se pelo momento de processamento, alocação de memória e tratamento de tipos:

Diretiva `#define`

- É uma instrução processada pelo **pré-processador** antes da etapa de compilação do código.
- Realiza a substituição puramente textual de todas as ocorrências do identificador pelo valor literal definido.
- **Uso de memória:** Não consome bytes da memória SRAM, pois seu valor é embutido diretamente como constante literal nas instruções binárias da memória Flash.
- **Sintaxe e regras:** Não utiliza sinal de atribuição (`=`), não termina com ponto e vírgula (`;`), não possui tipagem de dados e não obedece a regras de escopo de variáveis.
- A diretiva `#define` é convencionalmente declarada **no topo do sketch**, ou seja, na área de escopo global antes das funções `setup()` e `loop()` (geralmente logo abaixo das inclusões de bibliotecas com `#include`)
- **Exemplo:** Ao encontrar `#define PINO_LED 13`, o pré-processador percorre o código-fonte e troca qualquer token isolado `PINO_LED` pela sequência de caracteres `13` antes de repassar o arquivo intermediário ao compilador.

Qualificador `const`

- É um qualificador aplicado a uma declaração formal de variável gerenciada diretamente pelo **compilador**.
- Declarar uma constante com `const` cria uma variável comum que possui tipo, escopo e endereço na memória, mas com uma trava de segurança ativada pelo compilador que impede que seu conteúdo seja alterado durante a execução do programa. Por se tornar imediatamente "somente leitura", ela precisa

receber seu valor definitivo no exato instante em que é criada, evitando que o microcontrolador trave seu espaço com lixo de memória e protegendo parâmetros críticos contra alterações acidentais ao longo do código.

- **Uso de memória:** Por padrão, comporta-se como uma variável convencional que aloca espaço na memória SRAM, salvo quando otimizada pelo compilador.
 - **Sintaxe e regras:** Utiliza sintaxe padrão de variável (com tipo, `=`, valor e `;`), respeitando a verificação estrita de tipos e o escopo do bloco onde foi definida.
 - **Exemplo:** `const byte ledPin = 13;`
-

Bloco 3 — Operadores Matemáticos, Lógicos, de Atribuição e Precedência

3.1 Operadores Aritméticos e Divisões

Os operadores aritméticos realizam cálculos matemáticos sobre valores numéricos de variáveis e constantes:

- `+` (Adição)
- `-` (Subtração)
- `*` (Multiplicação)
- `/` (Divisão)
- `%` (Módulo ou Resto da Divisão) — **Aplicável unicamente para operandos do tipo inteiro** (`int` ou `char`).

A Armadilha da Divisão Inteira:

Em C, se os dois números operados forem inteiros, o compilador realizará uma divisão inteira por padrão, **descartando completamente a parte fracionária do quociente** (sem realizar arredondamentos).

```
int resultado1 = 5 / 2; // O resultado gravado na variável será 2.
```

```
float resultado2 = 5 / 2; // O resultado continuará sendo 2.0 (pois a divisão entre inteiros ocorreu antes)!
```

```
float resultado3 = 5.0 / 2.0; // O resultado correto será 2.5 (pois ao menos um dos valores foi forçado como float).
```

3.2 Operadores Relacionais e Lógicos

Os operadores relacionais e lógicos funcionam como as **perguntas de tomada de decisão** do microcontrolador: eles analisam condições no circuito e respondem obrigatoriamente de forma binária com **1 (Verdadeiro)** ou **0 (Falso)**.

Operadores Relacionais: As Comparações de Grandeza Eles comparam dois valores numéricos entre si para checar igualdade ou escala matemática:

- **`==` (Igual a):** Pergunta se os dois lados são idênticos (ex.: `temperatura == 100`). *Atenção:* não confunda com o `=` simples, que serve para guardar/atribuir um valor em uma variável.
- **`!=` (Diferente de):** Checa se os dados são distintos entre si.
- **`>` e `<` (Maior / Menor que):** Avaliam magnitudes simples (ex.: `leituraSensor > 500`).
- **`>=` e `<=` (Maior ou igual / Menor ou igual):** Avaliam limites numéricos incluindo a própria borda de teste.

Operadores Lógicos: Conectores de Decisão Complexa Eles combinam ou invertem o resultado de duas ou mais comparações relacionais para definir regras compostas:

- **&& (Operador E / AND):** Exigente. Só resulta em **1** se **todas as condições individuais** forem verdadeiras simultaneamente[cite: 1, 2]. Se apenas uma falhar, o resultado todo vira **0**.
- **|| (Operador OU / OR):** Flexível. Resulta em **1** se **ao menos uma das condições** for verdadeira. Só retorna **0** se todas as condições falharem.
- **! (Operador NEGAÇÃO / NOT):** O inversor. Ele inverte o valor lógico: se uma condição era verdadeira (1), ele a transforma em falsa (0), e vice-versa.

Exemplo Prático

Imagine um sistema de estufa automatizada que só deve ligar a ventoinha de alívio se a temperatura estiver acima de **30°C** **E** a umidade passar de **80%**:

```
// O compilador resolve primeiro as comparações e depois conecta com o operador lógico
if (temperatura > 30 && umidade > 80) {
    digitalWrite(PINO_VENTOINHA, HIGH); // Executa apenas se o resultado do teste for 1
}
```

Se **temperatura** for 35 (Verdadeiro / 1) e **umidade** for 60 (Falso / 0), a expressão composta fica **1 && 0**. Pela regra do operador **&&**, a resposta final é **0**, e a ventoinha permanece desligada.

3.3 Incremento, Decremento e Atribuições Compactas

Os operadores de incremento, decremento e atribuição composta representam mecanismos sintáticos e operacionais em C projetados para condensar operações repetitivas, otimizar instruções de máquina e manipular dados diretamente em registradores.

Atribuição Composta (Operadores Abreviados) Em vez de repetir a variável receptora no lado direito de uma expressão matemática, o C funde o operador aritmético com o operador de atribuição:

- **Abreviações clássicas:** **+=**, **-=**, ***=**, **/=** e **%=**.
- **A armadilha da precedência oculta:** Em uma atribuição composta como **x *= y - 10;**, a regra do compilador C estabelece que toda a subexpressão à direita é computada antes da operação composta. Portanto, o compilador traduz como **x = x * (y - 10);**, e nunca como **x = x * y - 10;**.

Pré-Incremento (++x) versus Pós-Incremento (x++)

Ambos adicionam **1** à variável inteira, mas diferem no instante em que o novo valor se torna disponível para a expressão onde estão inseridos:

- **Pós-Incremento (x++ / Pós-fixado):** O compilador utiliza o valor corrente de **x** para avaliar ou atribuir na expressão e, como efeito colateral posterior, adiciona **1** à variável.

```
int x = 10;
int y = x++; // y recebe 10 (valor original); em seguida, x passa a valer 11
```

- Isso equivale sequencialmente a: **y = x;** seguido de **x = x + 1;**.

- **Pré-Incremento (++x / Prefixo):** O compilador incrementa o valor de x em 1 imediatamente e entrega o resultado já atualizado para o restante da expressão.

```
int a = 10;
```

```
int b = ++a; // a é incrementado para 11; b recebe imediatamente 11
```

- Isso equivale sequencialmente a: `a = a + 1;` seguido de `b = a;`.

3.4 Tabela de Precedência de Operadores

Quando uma mesma linha de comando possui múltiplos operadores, o compilador decide qual cálculo executar primeiro com base em uma hierarquia de prioridades fixa (regras de precedência). O uso de **parênteses ()** permite forçar e controlar a ordem em que os cálculos ocorrem.

Abaixo estão listados os níveis mais comuns de precedência na linguagem C (do de maior prioridade para o de menor):

1. ++ (Pré-incremento) e -- (Pré-decremento), Parênteses (), Elemento de array [].
2. ++ (Pós-incremento) e -- (Pós-decremento), Operador de negação lógica !.
3. * (Multiplicação), / (Divisão), % (Módulo ou Resto).
4. + (Adição), - (Subtração).
5. << , >> (Deslocadores de bits).
6. < , <= , > , >= (Operadores relacionais de magnitude).
7. == , != (Operadores relacionais de igualdade/diferença).
8. && (Operador lógico E).
9. || (Operador lógico OU).
10. = , += , -= , *= , /= , %= (Operadores de atribuição simples e compostas).

Resumo das Boas Práticas da Aula

1. **Sempre prefira o tipo de dado de menor tamanho possível** para economizar a preciosa memória SRAM do microcontrolador (ex: use `byte` para pinos).
 2. **Declare variáveis com escopo local** sempre que possível para que a memória SRAM seja liberada ao fim das funções.
 3. **Utilize #define** para associar nomes fáceis a pinos físicos do circuito sem desperdiçar memória RAM de variáveis.
 4. **Cuidado redobrado com a divisão inteira** e garanta que ao menos um elemento seja representado com ponto decimal (como `.0`) quando necessitar de resultados com casas fracionárias.
-